

DSA Day 2: Arrays (1-D & 2-D)

Book: Data Structures Through C in Depth - Ch2 §2.1 · pp 10-17 | Code in Python

Overview

Arrays store elements contiguously so element i is reached in $O(1)$ by $\text{address} = \text{base} + i * \text{size}$. 2-D arrays are stored row-major in C.

Key concepts to master

- Contiguous memory, $O(1)$ random access
- Address arithmetic $\text{base} + i * \text{size}$
- Row-major storage of 2-D arrays
- Insert/delete in middle is $O(n)$ (shifting)
- Passing arrays to functions

Python implementation

```
a=[5,2,9,1,7]
def reverse(a):
    i,j=0, len(a)-1
    while i<j: a[i],a[j]=a[j],a[i]; i+=1; j-=1
def transpose(m):
    return [[m[j][i] for j in range(len(m))] for i in range(len(m[0]))]
```

Complexity

Access $O(1)$; search $O(n)$; insert/delete middle $O(n)$.

Common pitfalls

- Off-by-one on the last index
- Forgetting 2-D is row-major when computing addresses

Practice

- Reverse an array in place
- Transpose a 3x3 matrix
- Address of $a[i][j]$ given base & size

